

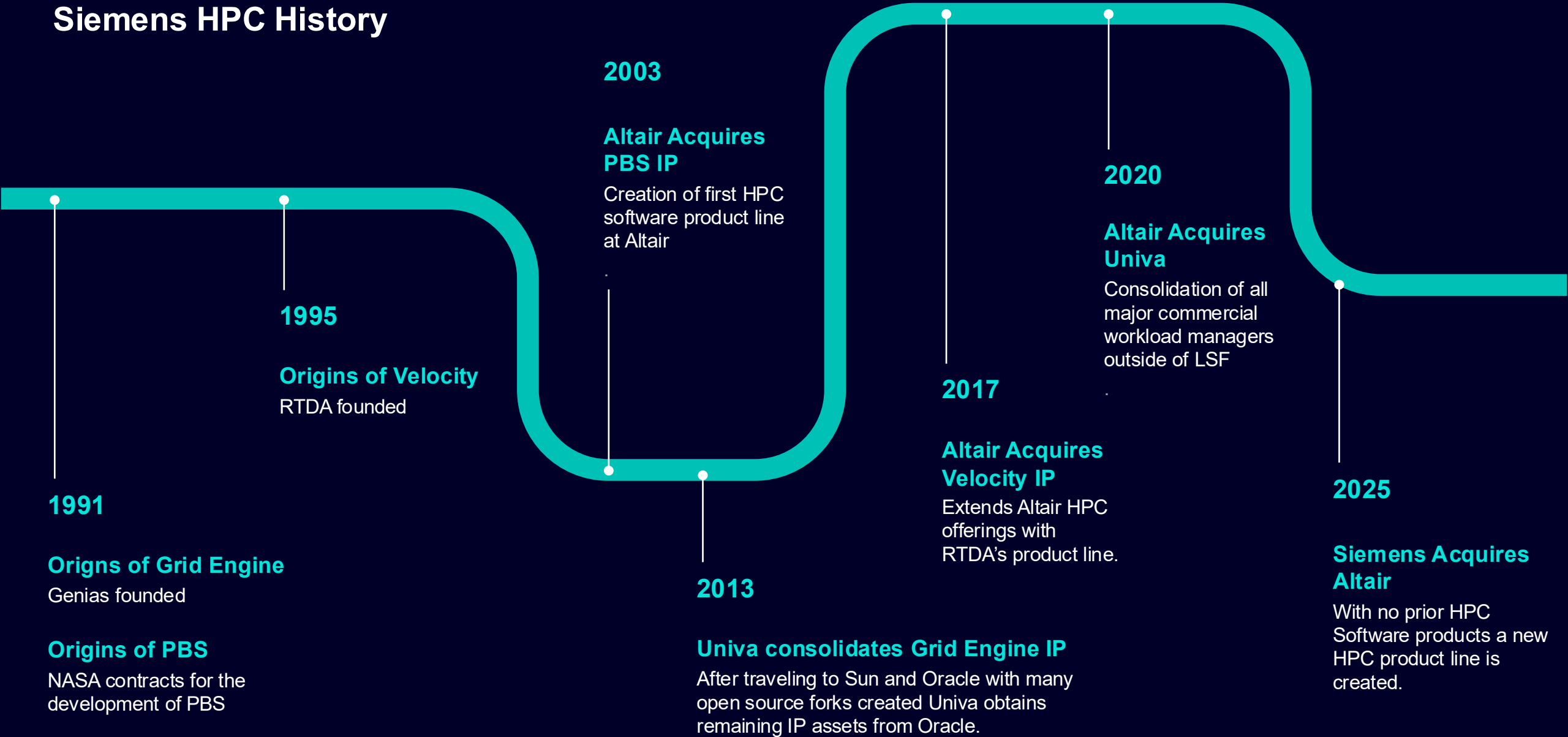


Enhancing the HPC-Quantum Interface: Scheduling, Orchestration, and Resource Management for Hybrid Workloads

OpenQSE July 2026

Cameron Brunner, Head of HPCWorks Development

Siemens HPC History





Get the right resources
to the right person (agent)
at the right cost
and at the right moment.

State of the Art: OpenPBS Hooks for QRM

How Quantum Resources Are Integrated Today

Current Integration Workflow

Step	What Happens
Step 1: User submits job	Job requests a quantum resource (e.g., quantum_resources=ibm_fez)
Step 2: Scheduler runjob hook	QRM hook acquires the QPU reservation before the job starts
Step 3: Job runs	Classical job executes; makes QRM API calls to the QPU for the life of the allocation
Step 4: Scheduler execjob_end hook	QRM hook releases the QPU reservation and performs cleanup

What Works Well

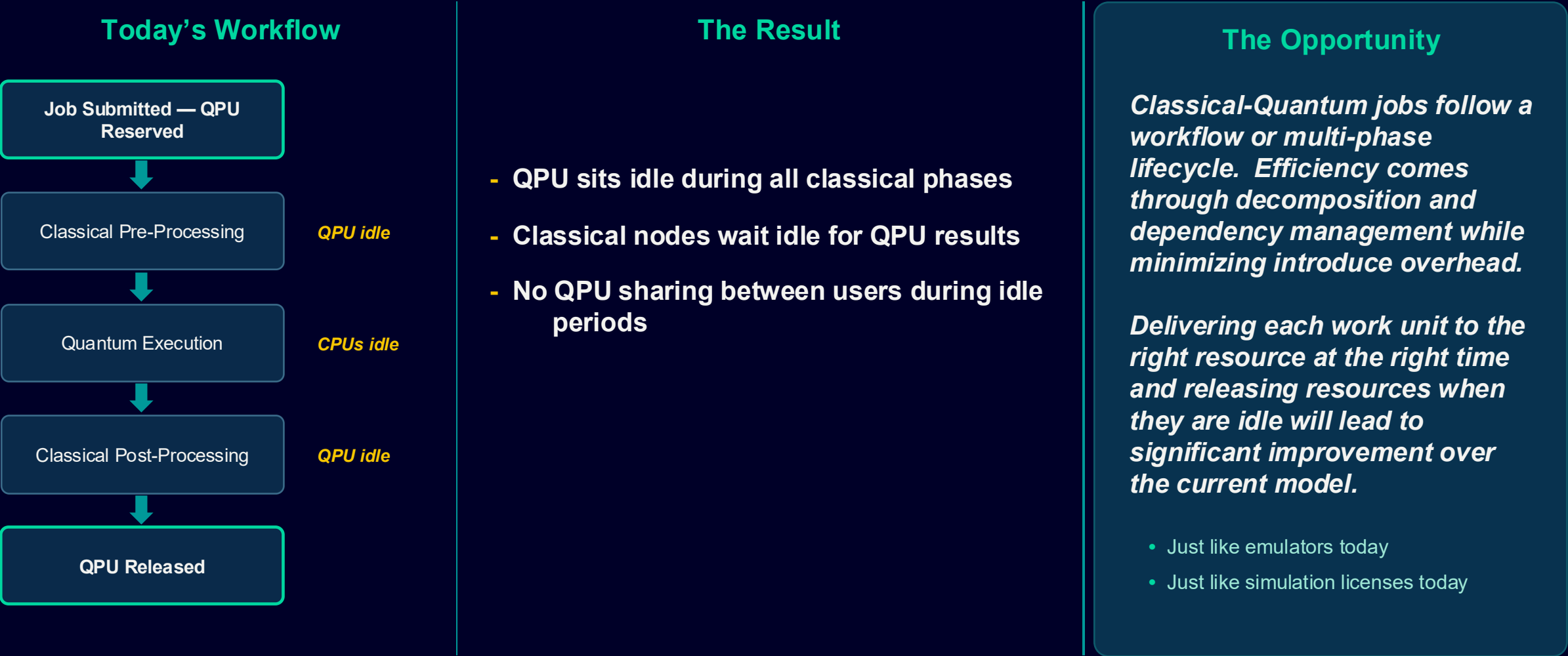
- ✓ Vendor-neutral abstraction
- ✓ Familiar job submission model for HPC users
- ✓ QPU reservation enforced before job dispatch

What Can Be Improved

- + Allocation granularity could be refined — releasing the QPU during classical phases would reduce idle time
- + Tighter classical-quantum coordination would improve overall resource utilization
- + Native support for iterative, shot-based workflows (e.g., VQE, QAOA) would unlock variational algorithm efficiency
- + Richer QPU resource descriptions (qubit count, topology, fidelity) would enable smarter scheduler placement
- + Abstracted resources would remove the need to specify a backend at submit time and facilitate pooling.

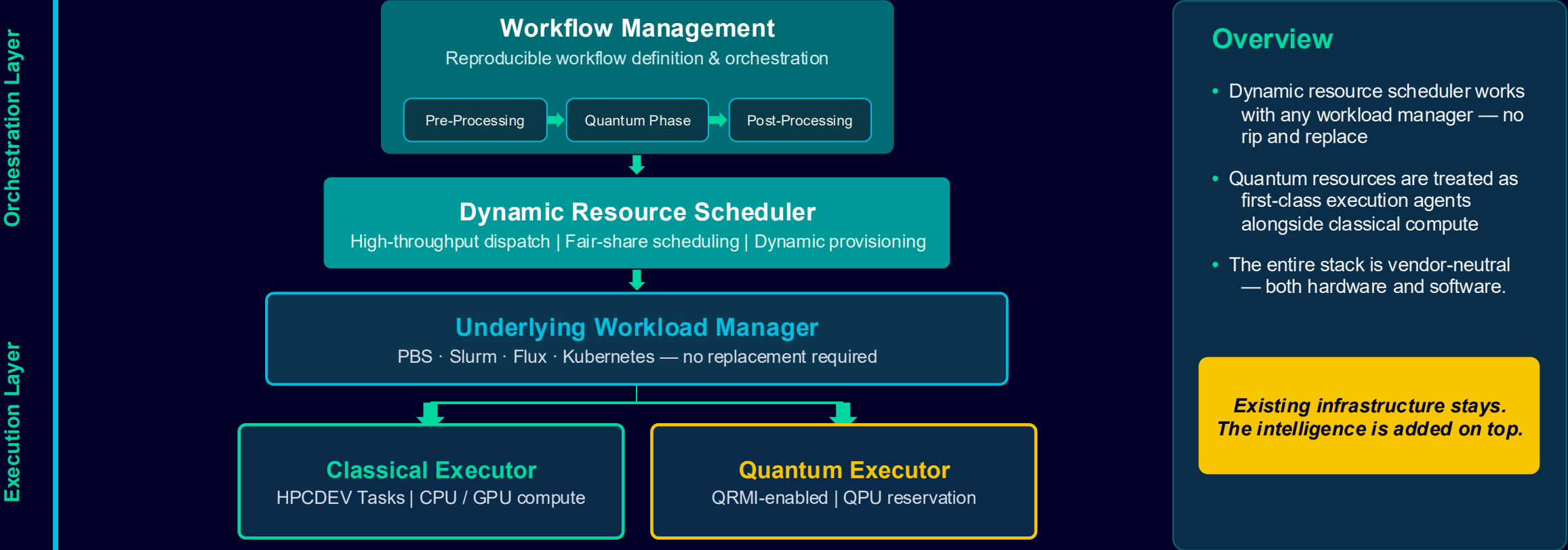
The Core Problem: Classical-Quantum Resource Waste

Why the Current Model Leaves Performance on the Table



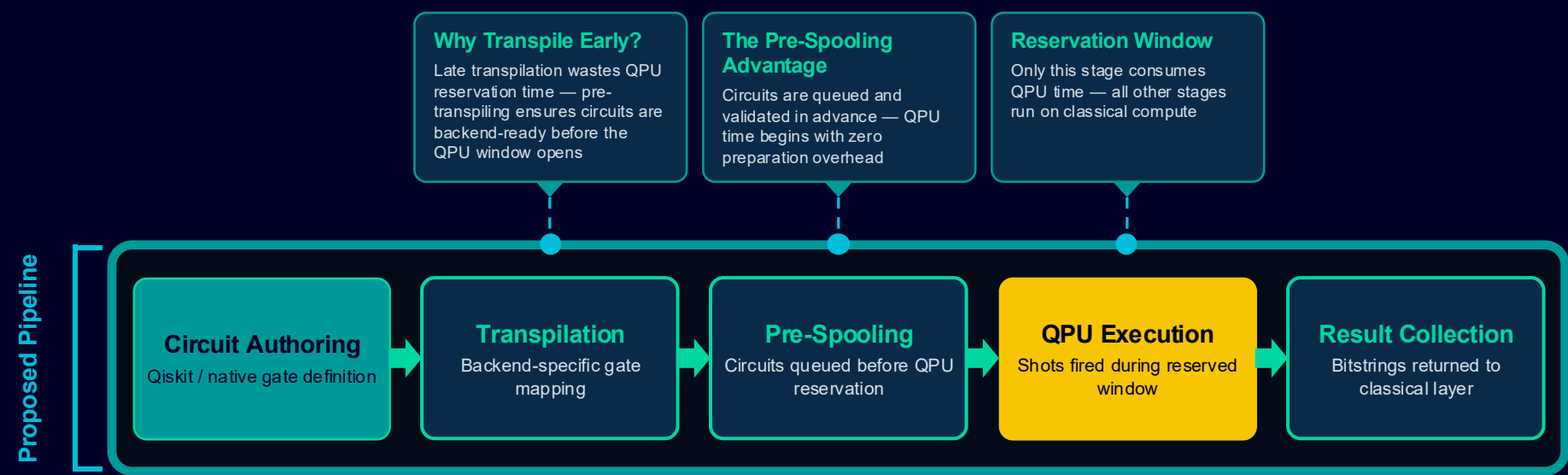
Decomposed Quantum-Classical Workflows

What HPC Workload and Workflow Managers Can Bring to Quantum



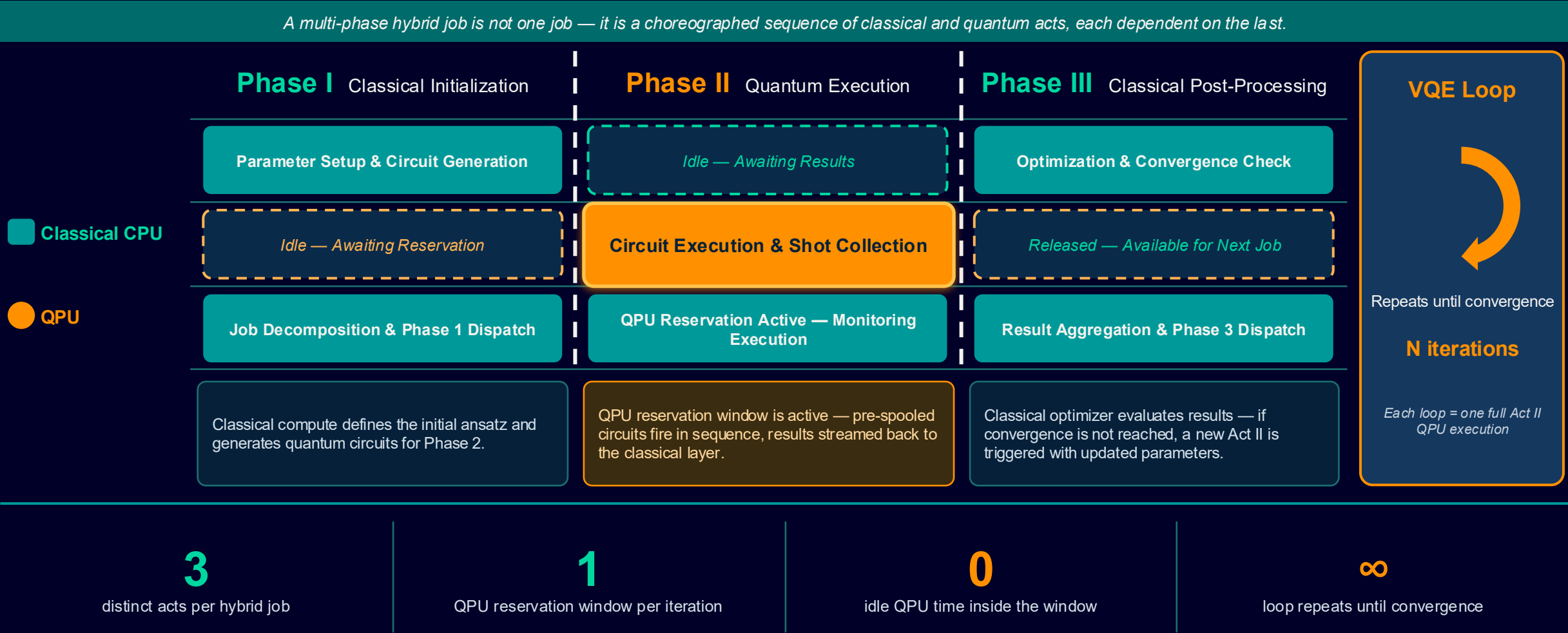
Queueing Efficiency: Pre-Spooled Quantum Execution

How Quantum Circuits Are Prepared, Queued, and Fired Before QPU Time Begins



Multi-Phase Jobs: Staged Quantum-Classical Execution

How Complex Hybrid Workloads Are Decomposed, Scheduled, and Orchestrated Across QPU and CPU



Micro Task Optimization(MTO): Batching Very Short Tasks

How MTO Eliminates Scheduler Overhead for High-Frequency Quantum Workloads

Without MTO: every shot batch pays full scheduler overhead. With MTO: one job, thousands of shots, near-zero overhead per batch.



1

Job submission per QPU reservation window

N

Batches executed within that single job

~0

Scheduler overhead per individual batch

QPU Resource Description: Beyond 'Give Me One QPU'

Why Schedulers Need Structured QPU Metadata to Make Intelligent Placement Decisions

A CPU scheduler knows cores, clock speed, and memory. A QPU scheduler must know qubits, topology, fidelity, and gate sets — or it is flying blind.

Resource Description Layer

REQUEST: QPU Resource

CURRENT STATE

RESOURCE TYPE: quantum
QUANTITY: 1
BACKEND: any
QUBITS: [not specified]
TOPOLOGY: [not specified]
GATE SET: [not specified]
FIDELITY: [not specified]
CONNECTIVITY: [not specified]
NOISE MODEL: [not specified]
CALIBRATION: [not specified]

The scheduler knows nothing about the QPU it is allocating. Any QPU is treated as interchangeable — circuit compatibility, fidelity requirements, and topology constraints are completely ignored.

⚠️ Blind Placement

⚠️ No Fidelity Guarantee

⚠️ Circuit Mismatch Risk

vs.

QPU RESOURCE DESCRIPTOR v1.0

PROPOSED STATE

RESOURCE TYPE: QPU
BACKEND ID: ibm_torino
QUBIT COUNT: 133
TOPOLOGY: heavy-hex
GATE SET: [CX, ECR, RZ, SX, X]
AVG T1: 189.4 μ s
AVG T2: 134.7 μ s
CNOT FIDELITY: 99.1%
READOUT ERR: 0.8%
CALIBRATION AGE: 4h 12m

The scheduler can now make intelligent placement decisions — matching circuit qubit requirements to available topology, validating gate set compatibility, and prioritising freshly calibrated backends.

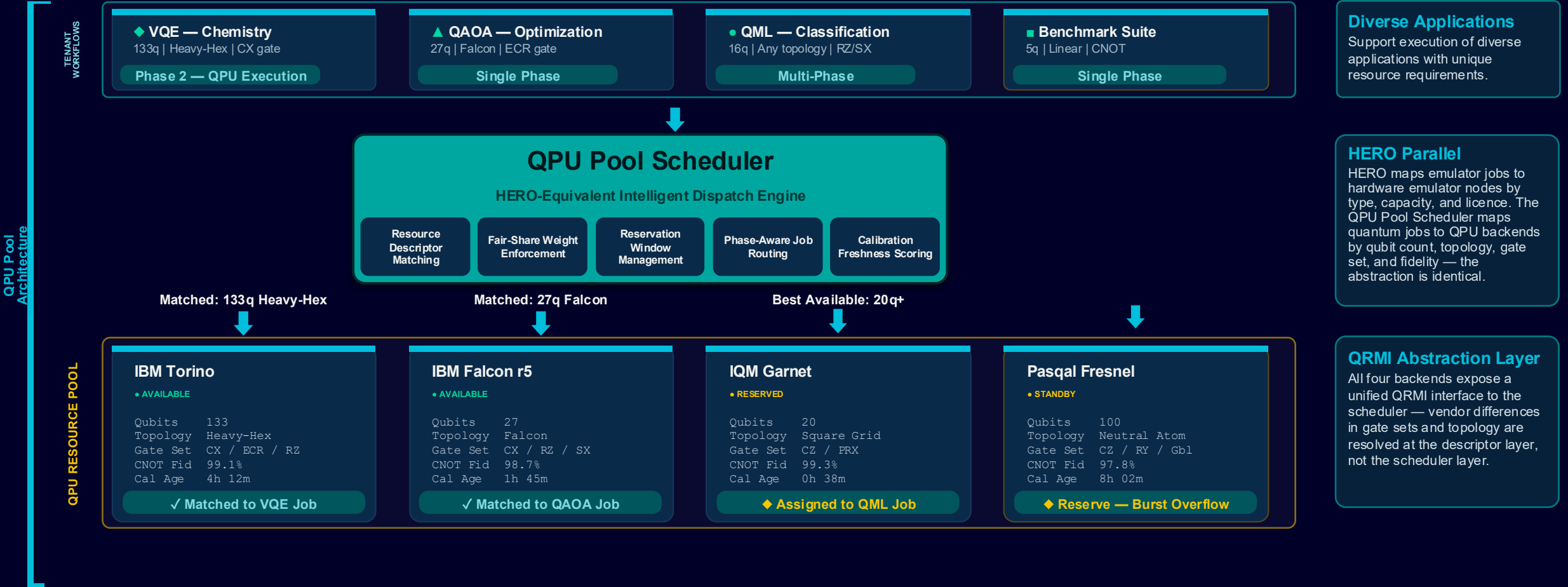
✅ Topology-Aware

✅ Fidelity-Matched

✅ Calibration-Fresh

QPU Pool Scheduling: A HERO-Equivalent Approach to Multi-QPU Resource Management

Extending HERO's Hardware Emulator Scheduling Principles to Heterogeneous Quantum Backend Pools



Thank you

Cameron Brunner

cameron.brunner@siemens.com

Head of HPCWorks Development

